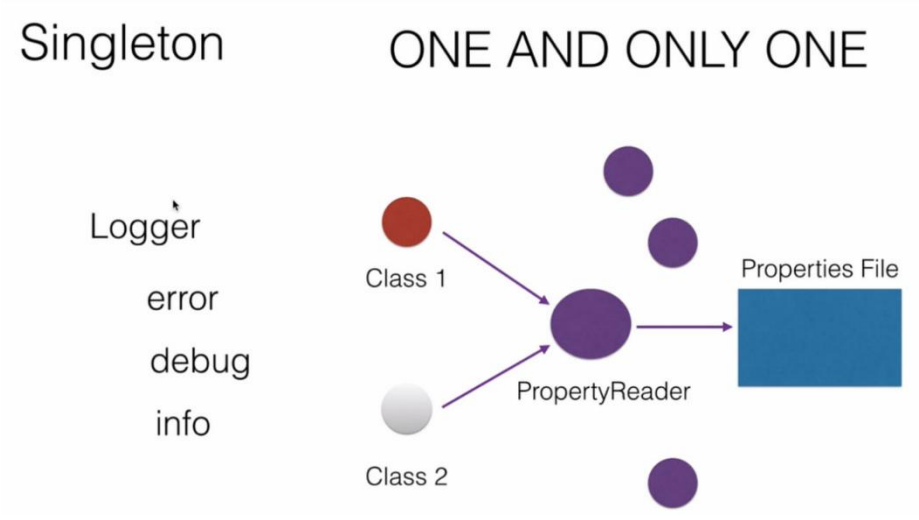
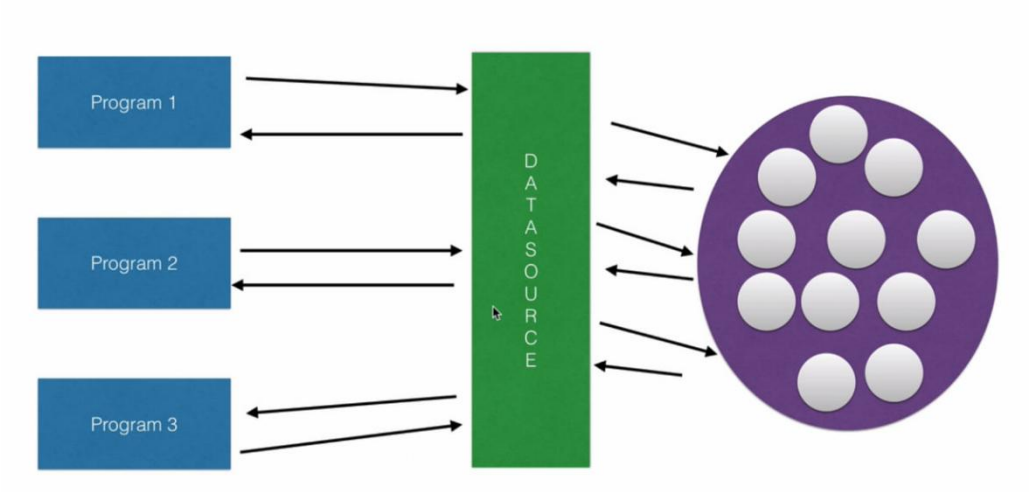
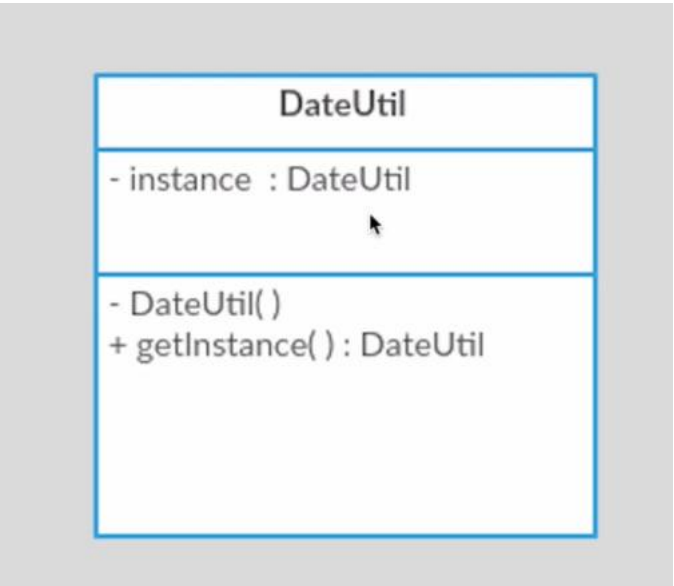
Singletone


Singleton
ONE AND ONLY ONE
Logger
error
debug
info
Class 1
Class 2
PropertyReader
Properties File


Program 1
Program 2
Program 3
DATASOURCE


DateUtil
- instance : DateUtil
- DateUtil( )
+ getInstance( ) : DateUtil

Steps

Declare the constructor of the class as private

Declare a static method

Declare a static member of the same class type in the class

1- Create Singleton class

Package Explorer

- corepattern
  - JRE System Library [jdk-25]
  - src
    - com.vibol.pattern.singletone
      - DateUtil.java
      - DateUtilTest.java
    - module-info.java
  - Java-8-to-Java-25

DateUtil.java

```java
package com.vibol.pattern.singletone;

public class DateUtil {

    // 3-Declare a static member of the same class type in the class

    private static DateUtil instance;


    // 1- when we create private constructor
    // no other class can not instancetiate
    private DateUtil() {

    }

    // 2- Declare a static method that return the instance of the DateUtil
    public static DateUtil getInstance() {

        if(instance != null) {
            instance = new DateUtil();
        }
        return instance;
    }


}
```

2- Test Instance the same create

Package Explorer

- corepattern
  - JRE System Library [jdk-25]
  - src
    - com.vibol.pattern.singletone
      - DateUtil.java
      - DateUtilTest.java
    - module-info.java
  - Java-8-to-Java-25

DateUtil.java | DateUtilTest.java

```java
package com.vibol.pattern.singletone;

public class DateUtilTest {

    public static void main(String[] args) {
        // create instance object
        DateUtil dateUtil1 = DateUtil.getInstance();
        DateUtil dateUtil2 = DateUtil.getInstance();

        // check the pointer point to the same memory
        System.out.println(dateUtil1 == dateUtil2);

    }

}
```

Problems | Javadoc | Declaration | Console

```
<terminated> DateUtilTest [Java Application] C:\Program Files\Java\jdk-25\bin\javaw.exe (Apr 19, 2026, 2:54:00 PM – 2:54:00 PM) [pid: 9380]
true
```

3- Eager Initialization បង្កើតមុន

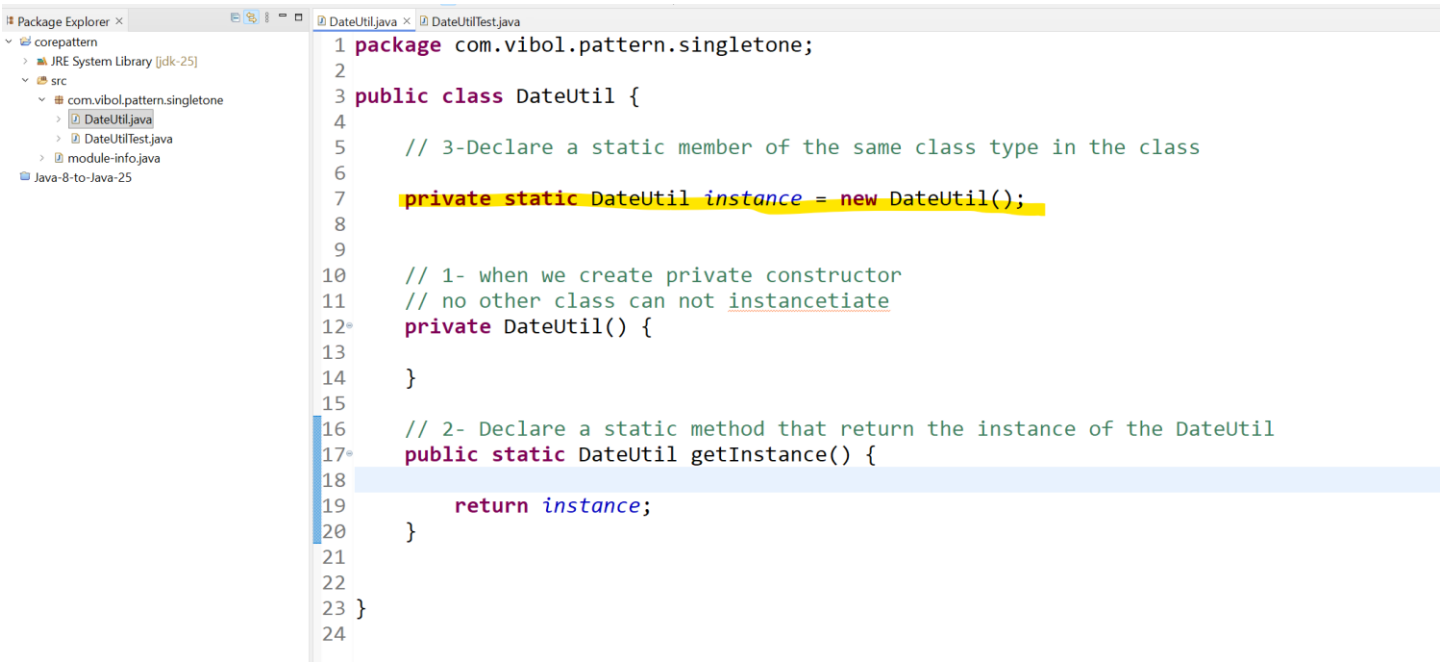
```java
// 2- Declare a static method that return the instance of the DateUtil
public static DateUtil getInstance() {

    if(instance != null) {
        instance = new DateUtil();
    }
    return instance;
}
```

This above called lazily creation : ពេលហៅបានវាបង្កើតអោយយើង

យើងបង្កើត Instance ពេលដែល getIntance invoke

- យើងអាចបង្កើតតាមលក្ខណៈ: Eager Initialization ពេលដែលហៅគឺអោយ Object

  នឹងបង្កើតតែម្តង



- របៀបមួយទៀត យើងអាចង្កើត Eager Initialization តាមរយៈ static block

```java
package com.vibol.pattern.singletone;

public class DateUtil {

    // 3-Declare a static member of the same class type in the class

    private static DateUtil instance;

    // static block Eager Initialization
    static {
        instance = new DateUtil();
    }


    // 1- when we create private constructor
    // no other class can not instancetiate
    private DateUtil() {

    }

    // 2- Declare a static method that return the instance of the DateUtil
    public static DateUtil getInstance() {

        return instance;
    }

```

Note : static block in java are excuted when the class is loaded into Memory

⇨ If you are pretty sure that your singleton instance will be used at least once in the Application because the instance will be created right when the class is loaded or when the class is being used

យើងប្រើវានៅពេលដែលដឹងថា application ត្រូវការបង្កើត Instance ពេលបើកមកដំបូង តែជាទូទៅគេប្រើប្រាស់ ជាលក្ខណៈ Lazy ពេលហៅបានអោយវាបង្កើត

4- Thread Safe

```
// 2- Declare a static method that return the instance of the DateUtil
public static DateUtil getInstance() {

    if (instance != null) {
        instance = DateUtil.getInstance();
    }

    return instance;
}
```

ពេលដែលយើងបង្កើត Instance តាមរបៀបនឹងវាអាចមានករណី Multi Thread that can make two instance in the same time to perveance this happening we use key work "**synchronized**" that will make the instance have only one object.

```
// 2- Declare a static method that return the instance of the DateUtil
public static synchronized DateUtil getInstance() {

        if (instance != null) {
            instance = DateUtil.getInstance();
        }

    return instance;
}
```

Or we can create block for thread :

```
// 2- Declare a static method that return the instance of the DateUtil
public static DateUtil getInstance() {

    synchronized(DateUtil.class) {
        if (instance != null) {
            instance = DateUtil.getInstance();
        }
    }

    return instance;
}
```

This synchronized block will confirm no other thread can access this block.

Improve Multi Threading :

DateUtil.java × DateUtilTest.java

```java
package com.vibol.pattern.singletone;

public class DateUtil {

    // 3-Declare a static member of the same class type in the class

    private static volatile DateUtil instance;

    // 1- when we create private constructor
    // no other class can not instance
    private DateUtil() {

    }

    // 2- Declare a static method that return the instance of the DateUtil
    public static DateUtil getInstance() {
        if (instance == null) {
            synchronized (DateUtil.class) {
                if (instance == null) {
                    instance = new DateUtil();
                }
            }
        }
        return instance;
    }

}
```

- Check the instance == null បើវាអត់មាន Object បានអោយវាបង្កើត បើវាមានហើយ គឺមិនអោយបង្កើតទេ
- Put word “volatile” បង្កើតអោយ Thread តែមួយវាអ្នកធ្វើការ

5- Handle with serialization and deserialization

```java
import java.io.FileInputStream;
import java.io.FileNotFoundException;
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;

public class DateUtilTest {

    public static void main(String[] args) throws FileNotFoundException, IOException, ClassNotFoundException {

        String url = "C:\\Users\\GE66\\Desktop\\Udemy-Course\\Udemy-Design Patterns\\1- Singleton\\file\\dateUtil.ser";

        DateUtil dateUtil1 = DateUtil.getInstance();

        ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream(url));
        oos.writeObject(dateUtil1);

        DateUtil dateUtil2 = null;
        ObjectInputStream ois = new ObjectInputStream(new FileInputStream(url));
        dateUtil2 = (DateUtil) ois.readObject();

        oos.close();
        ois.close();

        System.out.println(dateUtil1 == dateUtil2);

    }
```

Problems Javadoc Declaration Console

```
<terminated> DateUtilTest [Java Application] C:\Program Files\Java\jdk-25\bin\javaw.exe (Apr 19, 2026, 3:47:26 PM – 3:47:26 PM) [pid: 11856]
false
```

ពេលដែលយើងធ្វើការសរសេរ read and write file it created difference instance

5-1 Solution on serialization

ដោយសារតែ យើងចង់អោយ វា return object ដដែលចឹងយើងបង្កើត Object នៅក្នុង class អោយហើយ

Package Explorer

- corepattern
  - JRE System Library [jdk
  - src
    - com.vibol.pattern.si
      - DateUtil.java
      - DateUtilTest.java
    - module-info.java
- Java-8-to-Java-25

DateUtil.java × DateUtilTest.java

```java
public class DateUtil implements Serializable{

    private static final long serialVersionUID = 1L;

    // 3-Declare a static member of the same class type in the class

    private static volatile DateUtil instance;

    // 1- when we create private constructor
    // no other class can not instance
    private DateUtil() {

    }

    // 2- Declare a static method that return the instance of the DateUtil
    public static DateUtil getInstance() {
        if (instance == null) {
            synchronized (DateUtil.class) {
                if (instance == null) {
                    instance = new DateUtil();
                }
            }
        }
        return instance;
    }

    private Object readResolve() {
        return getInstance();
    }

}
```

Problems Javadoc Declaration Console ×

<terminated> DateUtilTest [Java Application] C:\Program Files\Java\jdk-25\bin\javaw.exe (Apr 19, 2026, 4:02:03 PM – 4:02:03 PM) [pid: 19440]

```
true
```

Java

```java
private Object readResolve() {
    return getInstance();
}
```

🔥 Why this matters

- Java serialization uses a **special method name**: `readResolve()` (exact spelling required).
- Because your method is misspelled (`readResovle`), Java **ignores it completely**.
- So when you deserialize, Java creates a **new object**, breaking your singleton.

ដើម្បីកុំអោយ class ដទៃធ្វើការ Access មកបានយើងអាចដាក់ថា

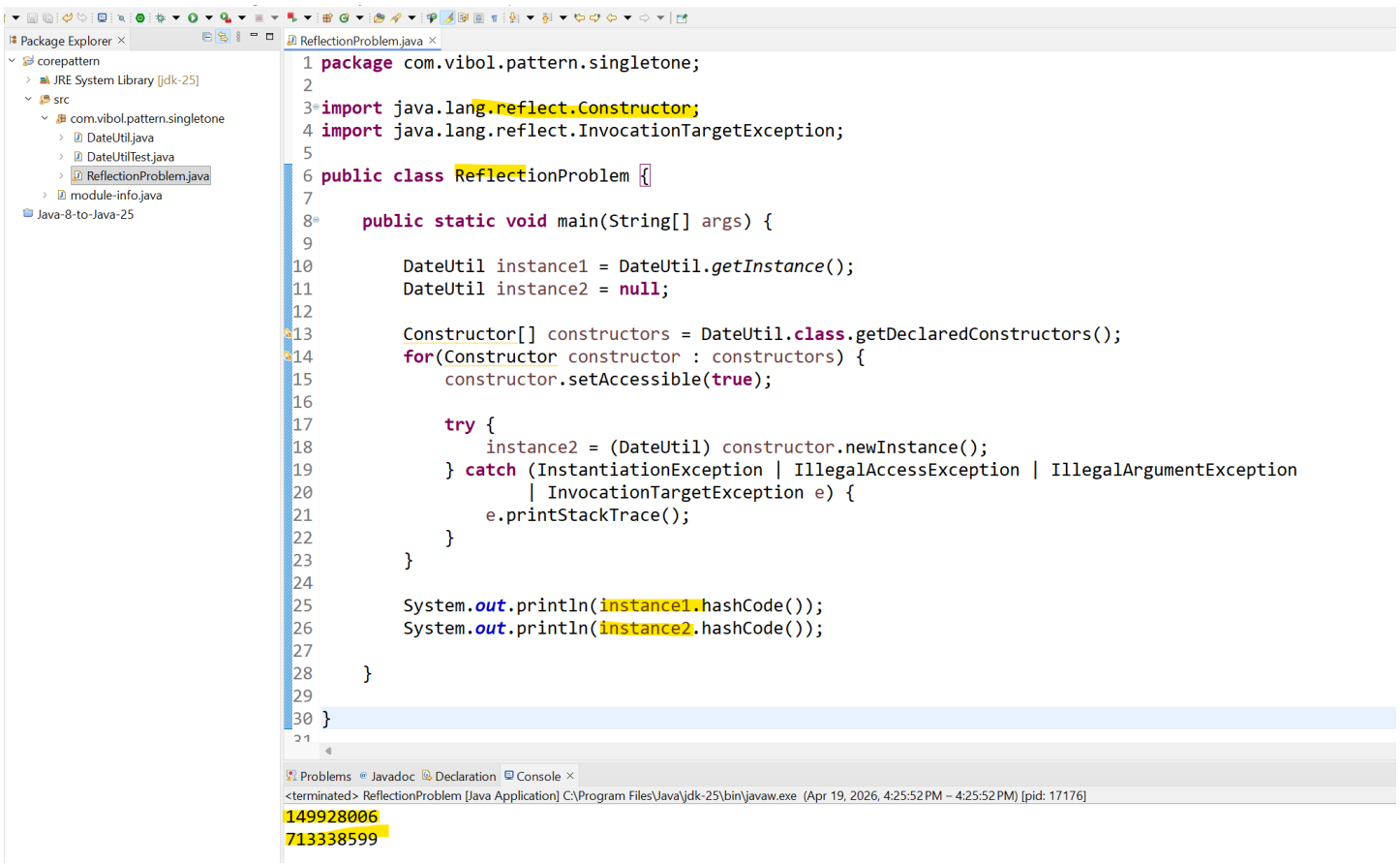
```
private Object readResolve() {
    return getInstance();
}

/*
protected Object readResolve() {
    return getInstance();
}
*/
```

6- Prevence clone object

Some client class want to clone our singletone but we don't want to we can set them not clone.

```
public class DateUtil implements Serializable, Cloneable{

    private static final long serialVersionUID = 1L;

    private static volatile DateUtil instance;


    private DateUtil() {

    }

    public static DateUtil getInstance() {
        if (instance == null) {
            synchronized (DateUtil.class) {
                if (instance == null) {
                    instance = new DateUtil();
                }
            }
        }
        return instance;
    }

    private Object readResolve() {
        return getInstance();
    }

    protected Object clone() throws CloneNotSupportedException{
        throw new CloneNotSupportedException();
    }

}
```

7- Reflection Problem

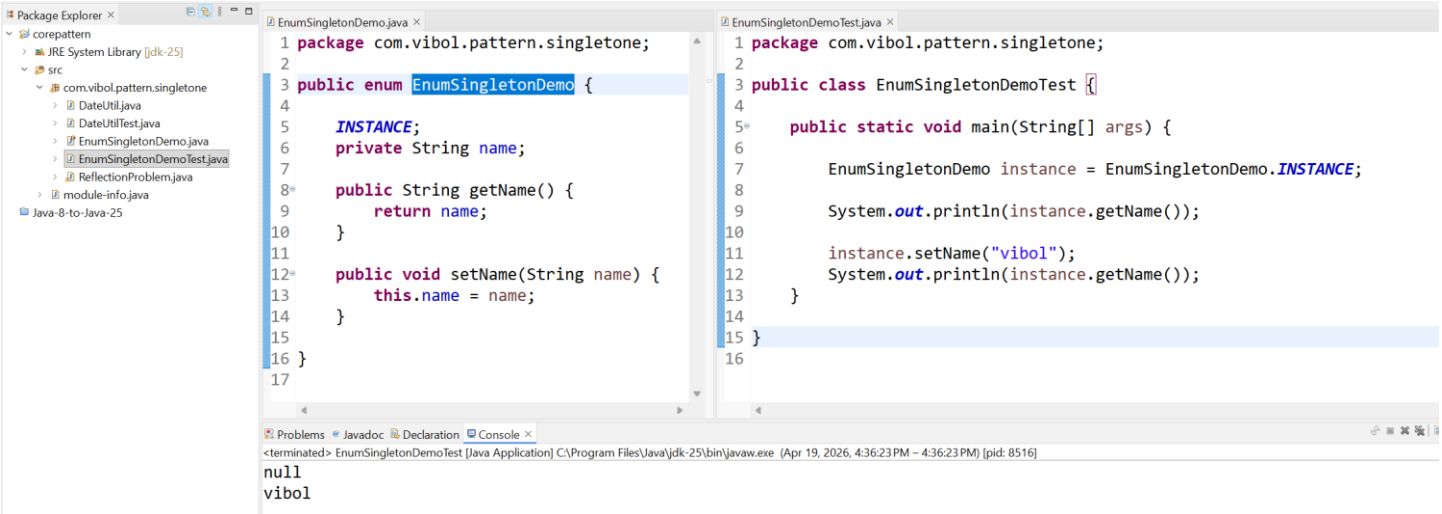



```java
package com.vibol.pattern.singletone;

import java.lang.reflect.Constructor;
import java.lang.reflect.InvocationTargetException;

public class ReflectionProblem {

	public static void main(String[] args) {

		DateUtil instance1 = DateUtil.getInstance();
		DateUtil instance2 = null;

		Constructor[] constructors = DateUtil.class.getDeclaredConstructors();
		for(Constructor constructor : constructors) {
			constructor.setAccessible(true);

			try {
				instance2 = (DateUtil) constructor.newInstance();
			} catch (InstantiationException | IllegalAccessException | IllegalArgumentException
				| InvocationTargetException e) {
				e.printStackTrace();
			}
		}

		System.out.println(instance1.hashCode());
		System.out.println(instance2.hashCode());

	}

}
```

ដោយប្រើប្រាស់ Reflection យើងអាចធ្វើការបង្កើត Instance ថ្មីបានសម្រាប់ Singletone

8- Solve the problem with from Reflection

យើងប្រើប្រាស់ Enum ដោយសារតែ Enum in the Java it doesn’t have constructor so we can not create break it by Reflection



យើងឃើញថា វាមិនអាច Set new name បានទេ ដូចនេះ យើងអាចបង្កើត singletone from Enum បានដូចគ្នា

Code :

1- DateUtile

```
package com.vibol.pattern.singletone;

import java.io.Serializable;

public class DateUtil implements Serializable, Cloneable {

	private static final long serialVersionUID = 1L;

	private static volatile DateUtil instance;


	private DateUtil() {

	}

	public static DateUtil getInstance() {
		if (instance == null) {
			synchronized (DateUtil.class) {
				if (instance == null) {
					instance = new DateUtil();
				}
			}
		}
		return instance;
	}

	private Object readResolve() {
	    return getInstance();
	}

	protected Object clone() throws CloneNotSupportedException {
		throw new CloneNotSupportedException();
	}

}
```

2- DateUtilTest

```
package com.vibol.pattern.singletone;

import java.io.FileInputStream;
import java.io.FileNotFoundException;
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;

public class DateUtilTest {

	public static void main(String[] args) throws FileNotFoundException, IOException, ClassNotFoundException {

		String url = "C:\\Users\\GE66\\Desktop\\Udemy-Course\\Udemy-Design Patterns\\1- Singleton\\file\\dateUtil.ser";

		DateUtil dateUtil1 = DateUtil.getInstance();

		ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream(url));
		oos.writeObject(dateUtil1);

		DateUtil dateUtil2 = null;
		ObjectInputStream ois = new ObjectInputStream(new FileInputStream(url));
		dateUtil2 = (DateUtil) ois.readObject();

		oos.close();
		ois.close();

		System.out.println(dateUtil1 == dateUtil2);

	}

}
```

3- Enum

```
package com.vibol.pattern.singletone;

public enum EnumSingletonDemo {

	INSTANCE;
	private String name;

	public String getName() {
		return name;
	}

	public void setName(String name) {
		this.name = name;
	}

}
```

4- Test Enum

```
package com.vibol.pattern.singletone;

public class EnumSingletonDemoTest {

	public static void main(String[] args) {

		EnumSingletonDemo instance = EnumSingletonDemo.INSTANCE;

		System.out.println(instance.getName());

		instance.setName("vibol");
		System.out.println(instance.getName());
	}

}
```